

URL shortener: tests implemented

552 tests in three suites, plus CI checks, load tests and chaos drills.

At a glance	
Total	552 tests in three suites; parametrized cases counted individually
Coverage	100% line and branch, gated separately for urlshort, sdhc and urlshort/adapters
Functional	All 32 acceptance criteria traced to passing tests (tests name their criteria)
Python	Python 3.11 and 3.12 on every pull request

1. Service tests: tests/service (364 tests, no servers needed)

Unit tests of single modules, and functional tests that drive the real HTTP API.

File	Tests	What it verifies
test_api.py	62	End-to-end HTTP: create, redirect, details, takedown, error format, stats access, rate-limit headers, trusted proxies and spoofing, security headers, CORS, docs toggle, logging, 503 with Retry-After
test_validation.py	45	URL safety: schemes, SSRF (private and loopback IPs, .internal), credentials, shorteners, blocklist, alias and TTL rules
test_config.py	39	Config file plus environment loading, type checks, unknown keys, secrets env-only, _FILE secrets, every settings section
test_analytics.py	24	Bot and browser detection, referrer domains, daily keyed visitor IDs (UTC, rotation), summaries
test_production.py	21	Each unsafe production setting refused, all reported at once, no bypass through code
test_events.py	18	Outbox atomicity, relay, idempotent aggregator, duplicates, dead-letter queue, retries, replay, schema upgrade
test_storage.py	18	SQLite repository, atomic counters, rollback, migrations from older schemas
test_clientip.py	16	Trusted-proxy rules including spoof attempts, IPv6 and malformed headers
test_service.py	16	Business rules without HTTP: idempotency, expiry, collisions, takedown, click recording, fail-open
test_logging.py	13	JSON format, redaction, request IDs surviving the background log thread, uvicorn JSON output
test_max_clicks.py	12	40 concurrent redirects on a 10-click link give exactly 10; bots do not consume the cap; fail-closed
test_ratelimit.py	12	GCRA burst and refill, client independence, eviction safety, exactness under 50 concurrent requests
test_auth.py	11	API keys: hashing, revocation, owner versus admin rights; invalid keys are 401, never anonymous
test_observability.py	11	Metrics by route template, metrics token, trace continuation, trace ID in logs, dashboards reference real metrics
test_audit.py	9	Hash-chain verification and tamper detection
test_lookalike.py	9	Mixed-script and punycode look-alikes rejected; single-script names accepted
test_errors.py	9	Domain error hierarchy: codes and HTTP statuses
test_admin_cli.py	5	Key management; signed checkpoints detect truncation and a full rewrite

File	Tests	What it verifies
test_codegen.py	5	Short-code generation
test_health.py, test_main.py, test_hourly.py, test_throttled.py	9	Probes; refused production start as one JSON line with exit code 2; clicks by hour; log throttling

2. Orchestrator tests: tests/sdlc (157 tests)

File	Tests	What it verifies
test_engine.py	35	Gates, retries, fallback, rollback, rework, re-planning, approvals, pause and resume, safe-stop, the stale-result race (deterministic)
test_policy.py	33	Every guardrail rule; this codebase's protected paths; agents cannot modify their own guardrails
test_llm.py	27	LLM client (HTTP faked at the connection), cassettes, strict answer validation, prompt-injection resistance, fallback through the real engine
test_foundations.py	17	Graph, context and lineage, audit log, metrics, approvals, git workspace
test_agents_paths.py	17	Change plans, recorded flawed drafts, materialised baselines, migration plans, review rounds
test_agents.py	14	Each agent's core behaviour: requirements, impact, design, review, QA tracing, security, docs, release
test_cli.py	9	run, resume, stop, verify; promote and its refusals (incomplete run, broken audit, dirty target, conflicts)
test_pipeline.py	5	All agents end to end on a copy of this repository, running the real service test suite

3. Infrastructure tests: tests/infra (31 tests, real Postgres and Redis)

Skipped locally without servers; run in CI's Infrastructure job, or with make ci-infra when Docker is running.

File	Tests	What it verifies
test_postgres.py	11	Same contract as SQLite; concurrent click caps; audit chain never forks; reversible migrations; API keys
test_redis.py	11	Shared rate limits; cache with takedown invalidation; Bloom filter; two app instances sharing state; fallback when Redis is down
test_events.py	8	Redis Streams; reclaiming a crashed consumer's messages; redirect to outbox to stream to stats; worker commands
test_resilience.py	1	Fail-fast 503s and the circuit breaker during a Postgres outage

4. Checks beyond the test suites

Check	Where it runs
Container smoke test (22 checks against the real Docker image)	CI integration job
Production mode refuses unsafe config and starts with safe config	CI integration job
Four SDLC scenarios end to end, with audit-chain verification	CI scenarios job
Lint, strict typing, security scans (bandit, pip-audit)	CI lint, type check and security jobs

Check	Where it runs
Load tests and chaos drills	scripts/; results in docs/evidence/

How the tests are organised

- Acceptance criteria are named in test docstrings (e.g. AC-MAXCLICKS-1); the QA agent and docs/requirements.md use the same mapping.
- Concurrency-sensitive code is tested under real parallel load: click caps, rate limiter, audit chain, orchestrator race.
- Failure paths are tested deliberately: storage errors, malformed input and events, outages, tampering, prompt injection.
- Every coverage exclusion is explicit and justified; no test exists only to raise coverage.